



Analogo meccanismo per gli altri array e variabili con la sola differenza dell'incremento da una cella all'altra in base al tipo di dato T, più precisamente in base a $\text{sizeof}(T)$.

Ogni successiva "nuova" struttura dati dichiarata (variabile o array che sia) viene memorizzata prima della precedente "vecchia". Quanto prima? Esattamente quanto basta perché le celle di memoria della struttura "nuova" finiscano all'indirizzo precedente della prima cella della struttura "vecchia".

Ad esempio, in riferimento al programma in figura 1, la variabile y occupa i 4 byte ($\text{sizeof}(\text{float})=4$) precedenti a quelli della variabile x. Vale a dire che $\&x = \&y + 1 * \text{sizeof}(\text{float}) = \&y + 4$. Ed è per questo che l'istruzione $(\&y)[1]=0$ presente alla riga 13 del mio codice "testo_es.cpp" sovrascrive il contenuto dei byte dedicati ad x, memorizzati subito dopo quelli di y, inserendo uno 0 al posto dell'1 inizialmente presente.